COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S1
DATE PERFORMED:	07/16/2026	**DATE SUBMITTED:**	07/21/2026
NAME:	Pangilinan Jr., Bienvenido B.	**INSTRUCTOR:**	Engr. Jimlord Quehado

ACTIVITY NO. 2 **[ACTIVITY NAME]: 2.1 Arrays, Pointers and Dynamic Memory Allocation**

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A: Implement static and dynamic memory allocation & ILO B: Create dynamically allocated objects using pointers and arrays

Table 2-1. Initial Driver Program

INPUT

```cpp
#include <iostream>
#include <string.h>

class Student{
private:
    std::string studentName;
    int studentAge;

public:
    //constructor
    Student(std::string newName ="John Doe", int newAge=18){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    };

    //deconstructor
    ~Student(){
        std::cout << "Destructor Called." << std::endl;
    }
    //Copy Constructor
    Student(const Student &copyStudent){
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }
    //Display Attributes
    void printDetails(){
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};
```

```
32
33  int main() {
34      Student student1( newName: "Roman", newAge: 28);
35      Student student2( student1);
36      Student student3;
37      student3 = student2;
38      return 0;
39  }
```

OUTPUT

```
Constructor Called.
Copy Constructor Called
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

OBSERVATION

Based on my observation, the **student1** already has its own place value which is “Roman” and “28”. When **student2** is created, it instantly copies the data from **student1** using the **Copy Constructor**, making **student1** and **student2** have the same data. After that, **student3** is created with default values and then, **student3** discards its default values and overwrites them by copying **student2** data using the **Copy Assignment Operator**. Finally, when the program ends, the **Destructor** is called for all three objects to safely clean up their memory.

Table 2-2. Modified Driver Program with Student Lists

INPUT

```cpp
#include <iostream>
#include <string>

class Student{
private:
    std::string studentName;
    int studentAge;

public:
    //constructor
    Student(std::string newName ="John Doe", int newAge=18){
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    };

    //deconstructor
    ~Student(){
        std::cout << "Destructor Called." << std::endl;
    }
    //Copy Constructor
    Student(const Student &copyStudent){
        std::cout << "Copy Constructor Called" << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }
    //Display Attributes
    void printDetails(){
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};
```

```cpp
int main() {
    const size_t j = 5;
    Student studentList[j] = {};
    std::string namesList[j] = {"Carly", "Freddy", "Sam", "Zack", "Cody"};
    int ageList[j] = {15, 16, 18, 19, 16};

    return 0;
}
```

OUTPUT

```
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

OBSERVATION

Based on my observation on this code, when the array **studentList** is created with a size of 5, the code automatically allocates memory and repeats the **Constructor to** call 5 times from the list to initialize each slot with default values. After the program finishes executing, the **Destructor** is automatically set when all in the list is called this will order to safely clear the objects from memory and clean up the system resources.

Table 2-3. Final Driver Program

INPUT

```cpp
#include <iostream>
#include <string>

class Student {
private:
    std::string studentName;
    int studentAge;

public:
    Student(std::string newName = "John Doe", int newAge = 18) {
        studentName = std::move(newName);
        studentAge = newAge;
        std::cout << "Constructor Called." << std::endl;
    }

    ~Student() {
        std::cout << "Destructor Called." << std::endl;
    }

    Student(const Student &copyStudent) {
        std::cout << "Copy Constructor Called." << std::endl;
        studentName = copyStudent.studentName;
        studentAge = copyStudent.studentAge;
    }

    void printDetails() {
        std::cout << this->studentName << " " << this->studentAge << std::endl;
    }
};

int main() {
    const size_t j = 5;
    std::string namesList[j] = { "Carly", "Freddy", "Sam", "Zack", "Cody"};
```

```cpp
    Student studentList[j];
    for (size_t i = 0; i < j; ++i) {
        studentList[i] = Student( namesList[i], ageList[i]);
    }
    Student studentCopyList[j] = {
        studentList[0], studentList[1], studentList[2], studentList[3], studentList[4]
    };
    for (size_t i = 0; i < j; ++i) {
        studentCopyList[i].printDetails();
    }

    return 0;
}
```

LOOP A

```
Student studentList[j];
for (size_t i = 0; i < j; ++i) {
    studentList[i] = Student( namesList[i], ageList[i]);
```

OBSERVATION

Student object with name and age values in each pass then it will call the **Constructor** and assigns the data into **studentList[i]**, and then immediately destroys the temporary object at the end of the loop.

LOOP B

```
for (size_t i = 0; i < j; ++i) {
    studentCopyList[i].printDetails();
```

OBSERVATION

In this part the program iterates through the **studentCopyList** then it calls the **printDetails()** resulting for the 5 students name from the list to get called in the output.

OUTPUT

```
"C:\Users\Bienvenido\CLio
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Constructor Called.
Destructor Called.
Constructor Called.
Destructor Called.
Constructor Called.
Destructor Called.
Constructor Called.
Destructor Called.
Constructor Called.
Destructor Called.
Copy Constructor Called.
Copy Constructor Called.
Copy Constructor Called.
Copy Constructor Called.
Copy Constructor Called.
Carly 15
Freddy 16
Sam 18
Zack 19
Cody 16
```

```
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
Destructor Called.
```

OBSERVATIONS
Based on the observation, the program begins by initializing the **studentList** array of 5 in which is called the default **Constructor** 5 times. The first for loop creates temporary Student objects that populated with values from **namesList** and **ageList** to overwrite each entry in **studentList**, as each temporary object is created then subsequently discarded, it triggers a **Constructor** then followed immediately by a **Destructor,** 5 times and then the second array of **studentCopyList** is created using array with elements from **studentList**, which is the by 5 times, as for the follow up, the second for loop calls **printDetails()** on each **studentCopyList**, printing the 5 students along with their respective ages starting from (**Carly 15, Freddy 16, Sam 18, Zack 19, Cody 16**) and lastly, the **Destructor** is automatically executed 10 times to clean up all 5 objects from **studentCopyList** and all 5 objects from **studentList** making sure the console will be clean up.

TTable 2-4. Modifications/Corrections Necessary

MODEFICATIONS

```
Student studentList[j];
Student* ptr = studentList;
for (size_t i = 0; i < j; ++i) {
    *ptr = Student( namesList[i], ageList[i]);
    ptr++;
}
Student studentCopyList[j] = {
     studentList[0],  studentList[1],  studentList[2],  studentList[3],  studentList[4]
};
Student* copyPtr = studentCopyList;
for (size_t i = 0; i < j; ++i) {
    copyPtr->printDetails();
    copyPtr++;
}
```

OBSERVATION
The code was still same from the table 2.3 the only modifications I did is using the **pointer** then when I executed the code, the program creates an array of 5 default Student objects, then uses a **pointer** to update each student with custom names and ages (creating them and then immediately destroying 5 temporary), the next one it copies these 5 students into a new array using the **copy constructor**, and uses a second pointer (**copyPtr**) to display the student details and when the program ends, the **destructors** automatically run to safely clean up all student objects from memory.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

INPUT

```cpp
#include <iostream>
#include <string>
class Item {
protected:
    std::string name;
    double price;
    int quantity;
public:
    Item(std::string n, double p, int q) : name( n), price(p), quantity(q) {}
    virtual ~Item() {}
    double calculateSum() const {
        return price * quantity;
    }
    std::string getName() const {
        return  name;
    }
    virtual std::string getCategory() const = 0;

    void displayDetails() const {
        std::cout << getCategory() << ": " << name
                  << " | PHP " << price << " x " << quantity
                  << " = PHP " << calculateSum() << std::endl;
    }
};

class Fruit : public Item {
public:
    Fruit(std::string n, double p, int q) : Item( n, p, q) {
        std::cout << "Constructor called for " << name << std::endl;
    }

    ~Fruit() override {
        std::cout << "Fruit Destructor called for " << name << std::endl;
```

```
    std::string getCategory() const override { return "Fruit"; }
};

class Vegetable : public Item {
public:
    Vegetable(std::string n, double p, int q) : Item(n, p, q) {
        std::cout << "Constructor called for " << name << std::endl;
    }

    ~Vegetable() override {
        std::cout << "Vegetable Destructor called for " << name << std::endl;
    }

    std::string getCategory() const override { return "Vegetable"; }
};

double TotalSum(Item* list[], int count) {
    double total = 0;
    for (int i = 0; i < count; i++) {
        total += list[i]->calculateSum();
    }
    return total;
}
int main() {
    Item* GroceryList[4];
    GroceryList[0] = new Fruit( n: "Apple", p:10, q:7);
    GroceryList[1] = new Fruit( n: "Banana", p:10, q:8);
    GroceryList[2] = new Vegetable( n: "Broccoli", p:60, q:12);
    GroceryList[3] = new Vegetable( n: "Lettuce", p:50, q:10);
```

```cpp
int count = 4;
std::cout << "\n-- Grocery List --\n";
for (int i = 0; i < count; i++) {
    GroceryList[i]->displayDetails();
}
std::cout << "\nTotal: PHP " << TotalSum(GroceryList, count) << std::endl;
std::cout << "\n-- Removing Lettuce --\n";
for (int i = 0; i < count; i++) {
    if (GroceryList[i]->getName() == "Lettuce") {
        delete GroceryList[i];
        GroceryList[i] = GroceryList[count - 1];
        count--;
        break;
    }
}
std::cout << "\n-- Updated Grocery List --\n";
for (int i = 0; i < count; i++) {
    GroceryList[i]->displayDetails();
}
std::cout << "\nUpdated Total: PHP " << TotalSum(GroceryList, count) << std::endl;

for (int i = 0; i < count; i++) {
    delete GroceryList[i];
}

return 0;
```

OUTPUT

```
Constructor called for Apple
Constructor called for Banana
Constructor called for Broccoli
Constructor called for Lettuce

-- Grocery List --
Fruit: Apple | PHP 10 x 7 = PHP 70
Fruit: Banana | PHP 10 x 8 = PHP 80
Vegetable: Broccoli | PHP 60 x 12 = PHP 720
Vegetable: Lettuce | PHP 50 x 10 = PHP 500

Total: PHP 1370

-- Removing Lettuce --
Vegetable Destructor called for Lettuce

-- Updated Grocery List --
Fruit: Apple | PHP 10 x 7 = PHP 70
Fruit: Banana | PHP 10 x 8 = PHP 80
Vegetable: Broccoli | PHP 60 x 12 = PHP 720

Updated Total: PHP 870
Fruit Destructor called for Apple
Fruit Destructor called for Banana
Vegetable Destructor called for Broccoli

Process finished with exit code 0
```

OBSERVATION

The program creates a 4-item **GroceryList** containing fruit and vegetable, each object the **Constructor** upon calling, then through the list to display item details and uses the **TotalSum()** function to compute an initial total of PHP 1370. When Lettuce is removed, its allocated heap memory is freed using **destructor**, and the updated total is displayed as PHP 870, and finally, the remaining 3 items are deleted to make sure it is all clean in the end.

Provide the following:

Summary of lessons learned

- Throughout this activity, I implemented the Big Three (Destructor, Copy Constructor, and Copy Assignment Operator) along with dynamic memory allocation (new/delete) and virtual functions to safely manage polymorphic objects without memory leaks.

Analysis of the procedure

- The program created default objects then updated them using the big three operations to duplicate data before safely de-allocating memory using destructors in reverse creation order.

Analysis of the supplementary activity

- Based on the code, all items were stored in a single pointer array (Item*), allowing accurate total price calculations and then there is the smooth dynamic deletion of lettuce

Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement

- I managed to complete all the objectives and handle memory allocation, but I still got stuck at times because transitioning from Python to C++ is quite a headache with all the extra syntax and manual memory steps and I need to keep practicing C++ concepts again and eventually learning modern features to make easy in programming.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Python AI tutor**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Syntax explanation, purpose and understanding of the code**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI was used to clarify the understanding of the code and how it works.**

COMPUTER ENGINEERING

QUEZON CITY

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.